

Phase 1

Telugu Language Steps

1.1 Dataset Collection

The dataset was compiled using two approaches: manual collection and existing datasets.

A. Manual Collection

Textual data was collected through web scraping and targeted extraction from the following sources:

- **News Articles:** Telugu news outlets such as Sakshi, NTV, TV9, and Andhra Jyothy.
- **Legislative Transcripts:** Andhra Pradesh Budget 2024 session transcripts.
- **Podcast Transcripts:** Telugu podcasts such as *Raw Talks with VK*.
- **Bhashaverse (IIITH) Dataset:** Telugu source and target language data from Bhashaverse Parquet files.

B. Pre-existing Datasets

Existing Telugu corpora were also incorporated:

- **AI4Bharat IndicCorp:** Used the Telugu portion of the AI4Bharat IndicCorp dataset.

1.2 Cleaning the Telugu Corpus

All corpus cleaning and preprocessing steps were performed using regular expressions (regex):

1. **HTML Tags:** Removed all HTML tags from the text.
2. **URLs:** Removed URLs and web links.
3. **Unicode Normalization:** Normalized Unicode characters for consistency.
4. **Telugu Characters:** Retained only Telugu characters and removed non-Telugu characters.
5. **Spaces and Tabs:** Replaced multiple spaces and tabs with a single space.
6. **New Lines:** Converted multiple consecutive new lines into a single new line.
7. **Punctuation Spacing:** Added appropriate spacing around punctuation marks.
8. **Final Space Cleaning:** Removed extra spaces introduced during punctuation processing.

1.3 Pipeline Implementation

- Collected Telugu data from manual sources and AI4Bharat IndicCorpV2.
- Cleaned the text and converted it into sentence-level data.
- Added M and D source tags and distributed sentences into 256 MD5-based buckets.
- Processed 8 GB of IndicCorp data using streaming.
- Removed exact duplicate sentences within the buckets.
- Created 80/10/10 train, validation, and test splits.
- Trained 5K and 10K Unigram SentencePiece tokenizers.
- Evaluated both tokenizers on 5,000 validation sentences.
- Selected the 10K tokenizer based on the lower UNK rate and better fertility.
- Tokenized the training corpus, producing 863,817,731 tokens.

1.4 Why SentencePiece and Not BPE?

- **BPE:** BPE works by **frequency-based merging**. Since Telugu is a **morphologically rich language**, BPE does not explicitly consider morphological structure; it simply merges the most frequently occurring token pairs. As a result, some **morphologically segments may be lost**.
- **SentencePiece (Unigram):** Unigram considers **different possible segmentations** of a word and selects the **most probable segmentation**. This provides more flexibility in representing the different morphological components of Telugu words and can help **preserve their morphological structure**.

1.5 Language Statistics

Dataset Statistics	Count	Data Split	Count
Manual sentence candidates	23,272,604	Training	48,581,452 (80.00%)
IndicCorp sentence candidates	46,190,318	Validation	6,069,130 (9.99%)
Total sentence candidates	69,462,922	Test	6,076,540 (10.01%)
Duplicate sentences removed	8,735,800	Total	60,727,122
Total unique sentences	60,727,122		
Unique manual sentences	23,154,617	Tokenizer Statistics	Value
Unique IndicCorp sentences	37,572,505	5K vocabulary fertility	1.675
Manual word count	307,468,665	10K vocabulary fertility	1.441
IndicCorp word count	364,668,963	5K UNK rate	0.0
Total word count	672,137,628	10K UNK rate	0.0
		Selected vocabulary	10K
		Final training tokens	863,817,731
Estimated Token Counts (10K fertility = 1.441)			
Manual	$307,468,665 \times 1.441 \approx 443.2\text{M}$	IndicCorp	$364,668,963 \times 1.441 \approx 525.7\text{M}$
Combined estimate $\approx 968.9\text{M}$ tokens			

Nepali Language Steps

1. Dataset Collection

The dataset compilation process was broadly categorized into two approaches: manual data collection and the utilization of pre-existing datasets.

A. Manual Collection

- **News Articles:** Collected Nepali news articles from Setopati, covering topics such as politics, entertainment, and global news.
- **Large Text Corpus:** Collected raw, uncleaned Nepali large text files for further processing.

B. Pre-existing Data

- **Nepali IRIS Research Dataset:** Streamed data from the Nepali IRIS research dataset for corpus construction.

2. Text Cleaning

The Nepali corpus was cleaned using the following preprocessing steps. All corpus cleaning and preprocessing steps were performed using regex:

- Removed HTML tags and URLs from the text.
- Normalized Nepali text using Unicode NFC normalization.
- Kept only Nepali characters, spaces, numbers, and basic punctuation.
- Removed unwanted characters and extra tabs/spaces.
- Normalized multiple newlines into a single newline.
- Added spaces around punctuation marks for consistent tokenization.
- Removed unnecessary leading and trailing spaces.

3. Pipeline Implementation (Nepali)

- Collected Nepali data from manual sources and a Hugging Face Nepali corpus.
- Cleaned the text and converted it into sentence-level data.
- Added M and D source tags and distributed sentences into 256 MD5-based buckets.
- Processed 8 GB of Hugging Face corpus data using streaming.
- Removed exact duplicate sentences within the buckets.
- Created 80/10/10 train, validation, and test splits.

- Trained 5K and 10K Unigram SentencePiece tokenizers.
- Evaluated both tokenizers on 5,000 validation sentences.
- Selected the 10K tokenizer based on the lower UNK rate and better fertility.
- Tokenized the training corpus, producing 554,121,940 tokens.

4. Language Statistics

Dataset Statistics	Count	Data Split	Count
Manual sentence candidates collected	3,624,895	Training	20,447,821 (80.00%)
Downloaded sentence candidates collected	30,963,245	Validation	2,554,861 (10.00%)
Total sentence candidates collected	34,588,140	Test	2,556,788 (10.00%)
Duplicate sentences removed	9,028,670	Total	25,559,470
Total unique sentences	25,559,470	Tokenizer Statistics	Value
Unique manual sentences	2,705,965	5K vocabulary fertility	1.579
Unique downloaded sentences	22,853,505	10K vocabulary fertility	1.382
Manual word count	135,909,429	5K UNK rate	0.0
Downloaded word count	367,234,259	10K UNK rate	0.0
Total word count	503,143,688	Selected vocabulary	10K
		Final training tokens	554,121,940

Using the 10K tokenizer fertility of 1.382, the estimated token counts from the source word counts are:

Manual	$135,909,429 \times 1.382 \approx 187.8$ million tokens
Downloaded	$367,234,259 \times 1.382 \approx 507.5$ million tokens
Combined estimate	≈ 695.3 million tokens

The actual tokenized training corpus contains 554,121,940 tokens.

Phase 2

Transformer Architecture

Both the languages follow a similar transforemr architecture here are the specifics about each module

Token and Positional Embeddings

Token embeddings map each token to a learned 384-dimensional vector, while learned positional embeddings provide position information for a maximum sequence length of 512. The two embeddings are added element-wise and embedding dropout with a rate of 0.1 is then applied:

Why Learned Positional Embeddings?

Vaswani et al. (2017) found that learned and sinusoidal positional embeddings produce nearly identical results within the trained sequence length, differing mainly in extrapolation behavior beyond it sinusoidal embeddings can generalize to longer sequences, while in the case of learned embeddings we require chunking. Since our sentences were short enough and learned embeddings were simpler to implement so we used learned embeddings.

Transformer Block

- **Multi-head causal self-attention:** Each block applies multi-head self-attention using separate query, key, and value projections, with a causal mask to prevent attention to future tokens.
- **Feed-forward network:** A two-layer position-wise feed-forward network is used with GELU activation, expanding the dimension from $d_{\text{model}} = 384$ to $d_{\text{ff}} = 1536$ before projecting it back to 384.
- **Residual connections:** Residual connections are applied around both the attention and feed-forward sub-layers, allowing the input of each sublayer to be added back to its output.

- **Layer normalization:** Pre-normalization is used, where LayerNorm is applied before both the attention and feed-forward sublayers. This provides more stable training for deeper Transformer stacks.
- **Dropout:** Dropout with a rate of 0.1 is applied to the attention probabilities and to the outputs of both the attention and feed-forward sublayers.

Twelve such Transformer blocks are stacked sequentially, followed by a final LayerNorm before the output projection to the vocabulary.

Telugu Phase 2 model Configurations

The configuration used for the Telugu GPT-style Transformer is shown below:

Parameter	Value
Vocabulary Size (V)	10,000
Maximum Sequence Length	512
Model Dimension (d_{model})	384
Number of Transformer Layers	12
Number of Attention Heads	8
Feed-Forward Dimension (d_{ff})	1,536
QKV Bias	False
Dropout Rate	0.1

Table 1: Telugu Phase 2 Transformer configuration

Parameter Count Calculation

The total parameter count is:

Component	Calculation	Parameters
Token Embedding	$V \times d_{\text{model}} = 10,000 \times 384$	3,840,000
Positional Embedding	$\text{max_seq_len} \times d_{\text{model}} = 512 \times 384$	196,608
Per Transformer Layer (no bias on linear layers):		
QKV Projections	$3 \times d_{\text{model}}^2 = 3 \times 384^2$	442,368
Output Projection	$d_{\text{model}}^2 = 384^2$	147,456
FFN Layer 1	$d_{\text{model}} \times d_{\text{ff}} = 384 \times 1536$	589,824
FFN Layer 2	$d_{\text{ff}} \times d_{\text{model}} = 1536 \times 384$	589,824
LayerNorms (2, weight+bias)	$2 \times (2 \times d_{\text{model}}) = 2 \times 768$	1,536
Total per layer	$442,368 + 147,456 + 589,824 + 589,824 + 1,536$	1,771,008
All 12 layers	$12 \times 1,771,008$	21,252,096
Final LayerNorm	$2 \times d_{\text{model}} = 2 \times 384$	768
Output Projection (tied)	Shared with Token Embedding	0
Total Parameters		25,289,472

Table 2: Parameter count breakdown for the Telugu Phase 2 model ($\approx$ 25.29M parameters)

Analysis of Telugu Model

Loss Curves

The training loss decreases steadily from approximately 9.1 to 4.2, indicating that the model is progressively learning the training data. The validation loss also decreases initially and gradually stabilizes around 5.0, showing continued improvement in generalization with no significant increase in validation loss.

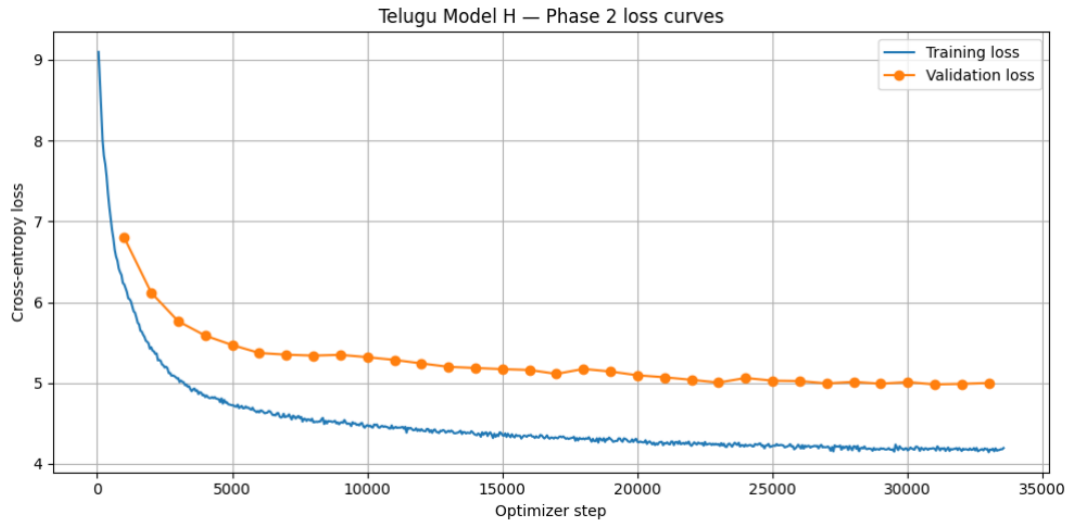


Figure 1: Training and validation loss curves for the Telugu model

Validation Metrics

A perplexity of close to 151 indicates the model is selecting from roughly 151 tokens in order to predict the next token, and the cross-entropy loss is also reasonably low on the held-out data, which indicates that the model is not randomly guessing over the text. The BPB of 0.66 suggests that the learned representations generalize decently on the unseen data.

Metric	Value
Cross-Entropy	5.020426
Perplexity	151.475794
Bits-per-Byte (BPB)	0.660157
Evaluated Tokens	1,638,400
Evaluated UTF-8 Bytes	17,975,776

Generation Evaluation

The generation results for different decoding strategies are shown below:

Strategy	BLEU-4	chrF	ROUGE-L	Distinct-1	Distinct-2	Repeated-3gram-%
Greedy	0.584938	10.985178	11.182346	0.096680	0.131944	0.847782
Temperature 0.5	0.807188	17.963811	10.067168	0.323242	0.558532	0.328629
Temperature 1.0	0.667935	22.400999	9.223221	0.654297	0.967262	0.008065
Temperature 1.5	0.341415	23.185740	3.659606	0.826172	0.999008	0.000000

Table 3: Generation evaluation across different decoding strategies

As the temperature increases, the generated text becomes increasingly diverse, as shown by the rise in Distinct-1 and Distinct-2. The repeated 3-gram rate decreases substantially, from 84.78% with greedy decoding to 0% at temperature 1.5, indicating reduced repetition. Temperature 0.5 achieves the highest BLEU-4 score of 0.8072, showing the highest similarity to the reference text among the tested strategies. Temperature 1.0 provides a better balance between similarity and diversity, with high Distinct-2 (0.9673) and very low repetition (0.81%). At temperature 1.5, diversity is highest but BLEU-4 and ROUGE-L decrease, indicating a trade-off between diversity and reference similarity.

Attention Analysis and Heatmaps

Across the 12 layers, attention distance and entropy both follow a similar trend, starting high, and then dropping sharply to a minimum around layer 6, and rising again toward the final layers. Layer 0 most of the attention weights are centered around zero and this is even evident from the entropy attention graphs where the initial layers have a very high entropy as compared to the previous layers. By layer 11, attention becomes more distributed and content-driven, attending to multiple relevant positions across the sequence. This progression

suggests the model first gathers coarse content, then specializes in morphological structure in the middle layers, and finally integrates this into longer-range dependencies useful for next-token prediction. Overall, the attention patterns reflect a well-formed and hierarchical representation of the Telugu input across depth.

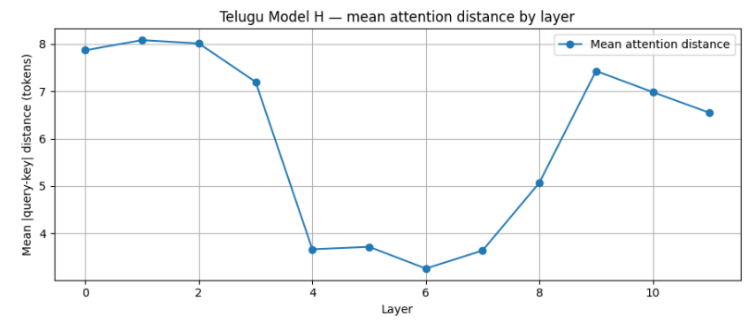


Figure 2: Mean attention distance by layer

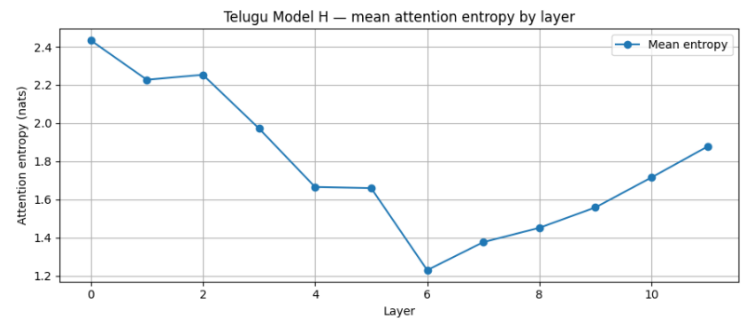


Figure 3: Mean attention entropy by layer

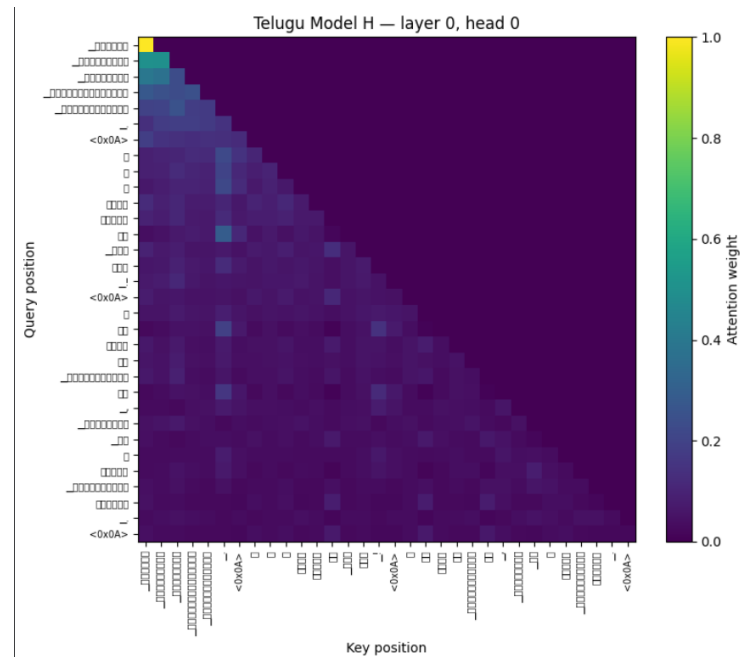


Figure 4: Attention heatmap — layer 0, head 0

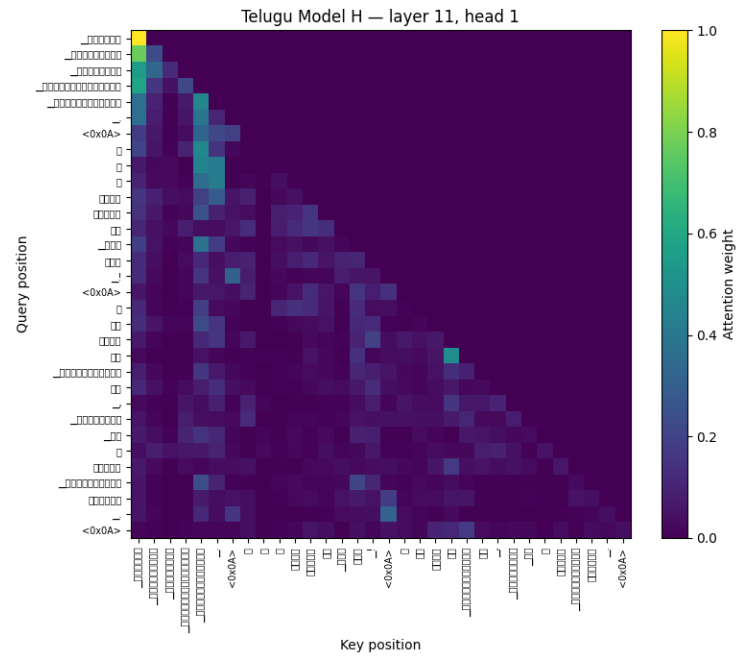


Figure 5: Attention heatmap — layer 11, head 1

Nepali Model Analysis

The Nepali GPT-style Transformer follows the same overall decoder-only Transformer architecture as the Telugu model, with a model dimension of 384, 12 Transformer layers, 8 attention heads, and a feed-forward dimension of 1536. The model uses learned absolute positional embeddings, pre-normalization, and tied input-output embeddings.

Parameter	Value
Vocabulary Size (V)	10,000
Maximum Sequence Length	512
Model Dimension (d_{model})	384
Number of Transformer Layers	12
Number of Attention Heads	8
Feed-Forward Dimension (d_{ff})	1,536
QKV Bias	False
Dropout Rate	0.1
Position Encoding	Learned Absolute
Normalization	Pre-Norm
Tied Embeddings	True

Table 4: Nepali model configuration

Analysis of Nepali Model

Loss Curves

The training loss decreases steadily from approximately 9.0 to 4.7, while the validation loss decreases from approximately 7.6 to 5.3. The continued decrease in both curves indicates that the model is learning progressively, although a gap between training and validation loss remains towards the end.

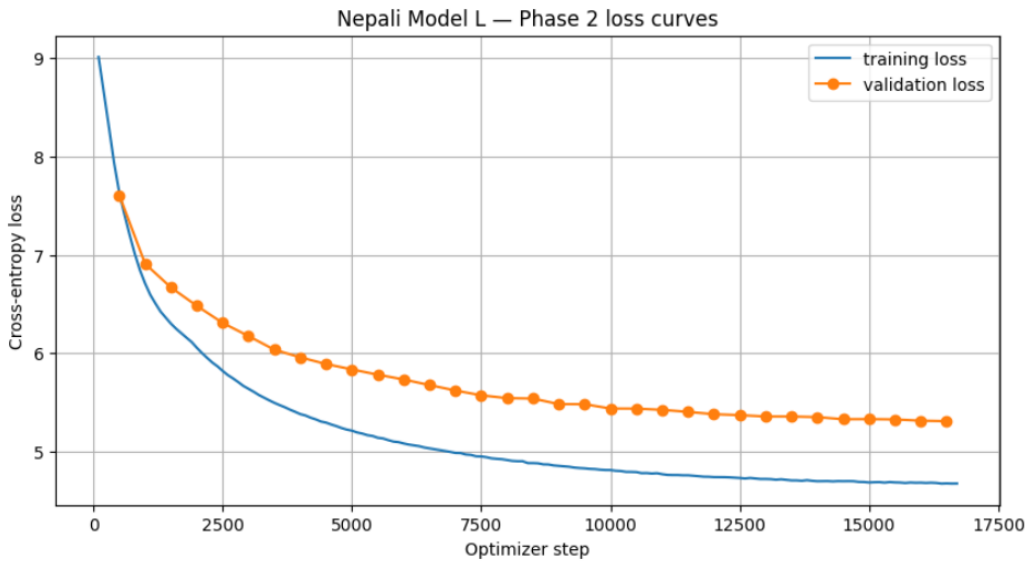


Figure 6: Training and validation loss curves for the Telugu model

Generation Evaluation

The generation results for different decoding strategies are shown below:

Strategy	BLEU-4	chrF	ROUGE-L	Distinct-1	Distinct-2	Repeated-3gram-%
Greedy	1.015664	15.830968	11.092134	0.157227	0.297619	0.621976
Temperature 0.5	0.964038	20.982782	11.057505	0.390625	0.752976	0.098790
Temperature 1.0	0.720886	20.941415	8.124324	0.678711	0.965278	0.002016
Temperature 1.5	0.445161	21.819186	3.119236	0.846680	1.000000	0.000000

Table 5: Generation evaluation across different decoding strategies for Nepali Model L

As the temperature increases, the generated text becomes more diverse, with Distinct-1 and Distinct-2 rising substantially. The repeated 3-gram rate drops sharply, from 62.20% with greedy decoding to 0% at temperature 1.5, indicating a large reduction in repetition. Greedy decoding achieves the highest BLEU-4 score of 1.015664, reflecting closer alignment with the reference text, while temperature 1.0 offers a strong balance between diversity and low repetition. At temperature 1.5, diversity peaks, but BLEU-4 and ROUGE-L decline, highlighting the trade-off between diversity and reference similarity.

Attention Analysis and Heatmaps

Similar to the Telugu model, the Nepali model’s mean attention distance and entropy both follow a similar trend across layers, starting high in the early layers, dropping to a minimum around layers 6–7, and rising again toward the later layers, though the increase in later layers is more moderate than in the Telugu model. Layer 0 gives attention weights mostly 0 evident by the violet distribution in the heatmap. By layer 11, attention becomes noticeably more distributed, with multiple distinct off-diagonal hotspots. This indicates that, as with Telugu, the Nepali model transitions from broad/sink-driven attention in early layers to focused local attention in the middle layers, and finally to more content-based, longer-range attention in the later layers.

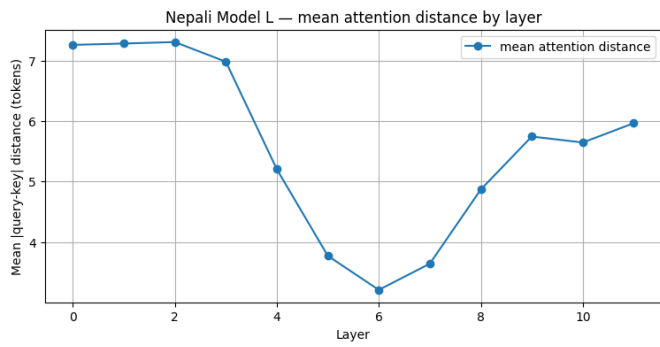


Figure 7: Nepali Model L — mean attention distance by layer

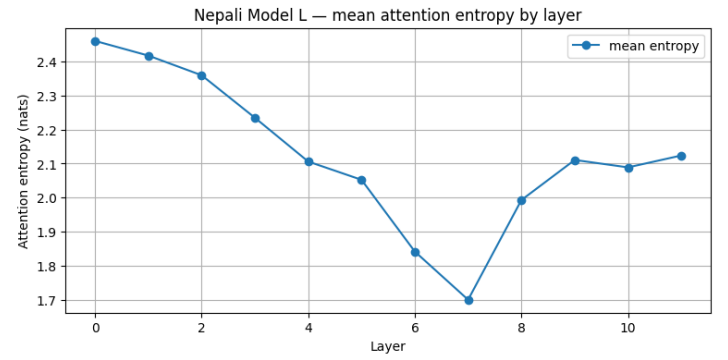


Figure 8: Nepali Model L — mean attention entropy by layer

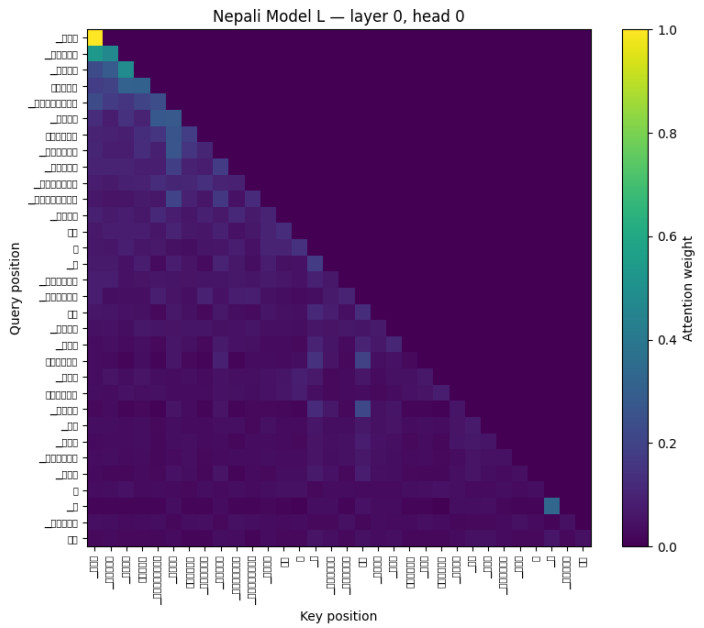


Figure 9: Attention heatmap — layer 0, head 0

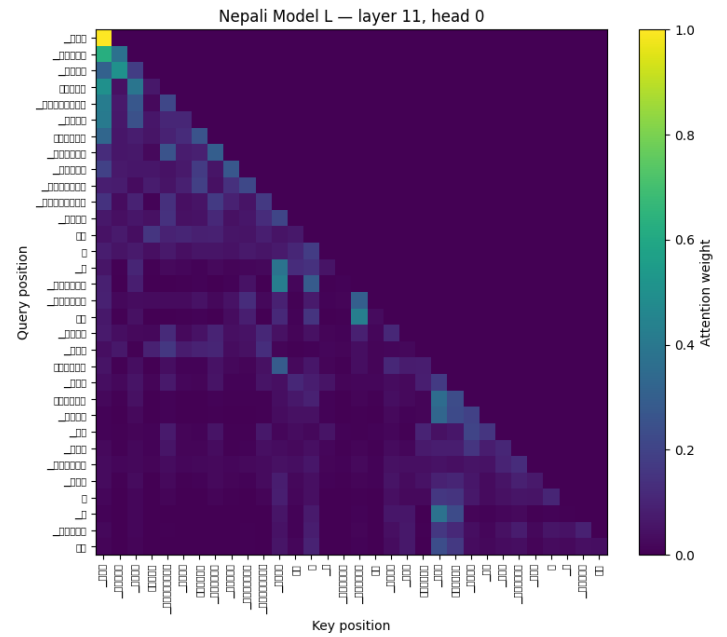


Figure 10: Attention heatmap — layer 11, head 0

Phase 3

Dataset Generation

Template	Description
T1: Direct Comparison	Compares two explicit numbers and selects the larger or smaller value.
T2: Greater/Smaller/Equal	Compares two numbers and outputs one of three balanced labels: Greater, Smaller, Equal.
T3: 3-Entity Transitive	Infers the relationship between endpoints of a three-entity chain using two hops.
T4: 4-Entity Multi-Hop	Reconstructs a four-entity chain from shuffled facts and performs three-hop reasoning.
T5: Multi-Hop with Distractors	Performs multi-hop reasoning while ignoring irrelevant facts about chain entities.
T6: Mixed Direction	Normalises forward and reverse relations before reasoning over a multi-hop chain.

Table 6: Six reasoning templates used for dataset generation.

The dataset was divided into six templates, as mentioned above. A total of 12,000 examples were generated, with 2,000 examples from each template. The dataset was then divided using an 80:10:10 split, resulting in 9,600 training, 1,200 validation, and 1,200 test examples. Each template contributed 1,600 examples to the training set, 200 examples to the validation set, and 200 examples to the test set.

Dataset Links

The Telugu Phase 3 dataset is available on Kaggle at: <https://www.kaggle.com/datasets/tanoojtadepalli/final-telugu-phase3>

The Nepali Phase 3 dataset is available on Kaggle at: <https://www.kaggle.com/datasets/tanoojtadepalli/nepali-phase3-dataset>

Shared Name Pool

Earlier, we trained the model using disjoint names across the training, validation, and test splits. However, this caused the model to treat the task primarily as a **name generalisation** problem rather than a reasoning problem. To address this, we use a shared pool of names across all three splits while varying only the underlying logical structure of the examples. This ensures that the main difference between the training and test sets is the reasoning problem itself. ““latex

How Data Leakage Across Splits Was Prevented

Two sets, `seen_prompts` and `seen_problem_keys`, are used to prevent data leakage. These sets are shared across the training, validation, and test splits. The `seen_prompts` set prevents the same prompt from appearing more than once. The `seen_problem_keys` set stores the template, attribute, ordered entities, question type, and numeric values for T1 and T2. It prevents the same logical problem from being reused even when the wording, order of facts, or distractors are changed. This ensures that the same underlying problem does not appear in different splits.

Analysis of Telugu model results

Loss Curves

The training loss decreased from 1.6022 to 0.1975 and finally to 0.1496 across the three epochs. The validation loss decreased from 0.2408 to 0.1539, but slightly increased to 0.1573 in the third epoch. Therefore, epoch 2 was selected as the best model based on validation loss. Overall, the loss shows a consistent reduction, indicating that the fine-tuned model is learning effectively. The slight increase in validation loss during the third epoch suggests that further training was not beneficial.

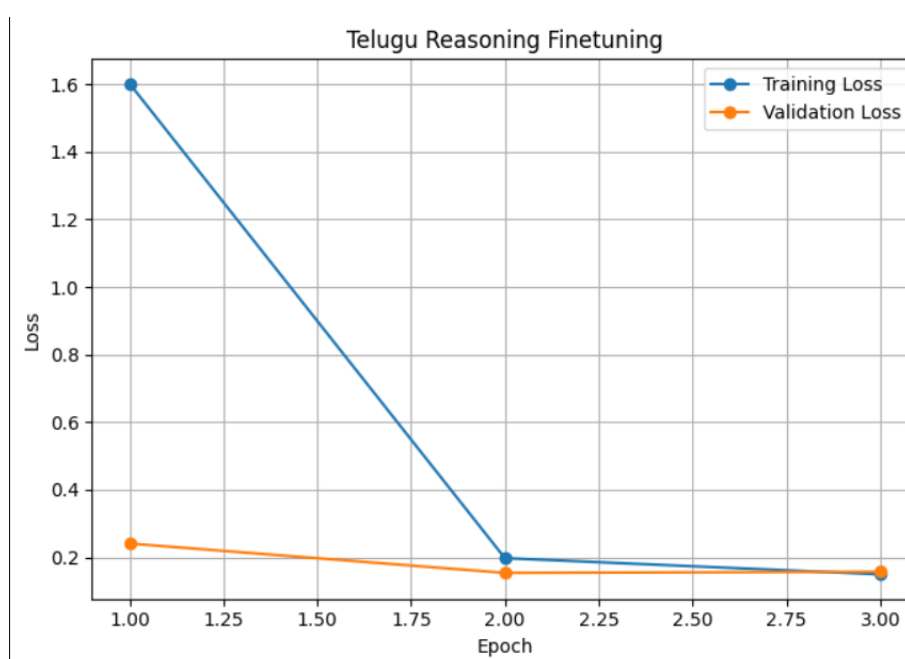


Figure 11: Training and validation loss curves during fine-tuning.

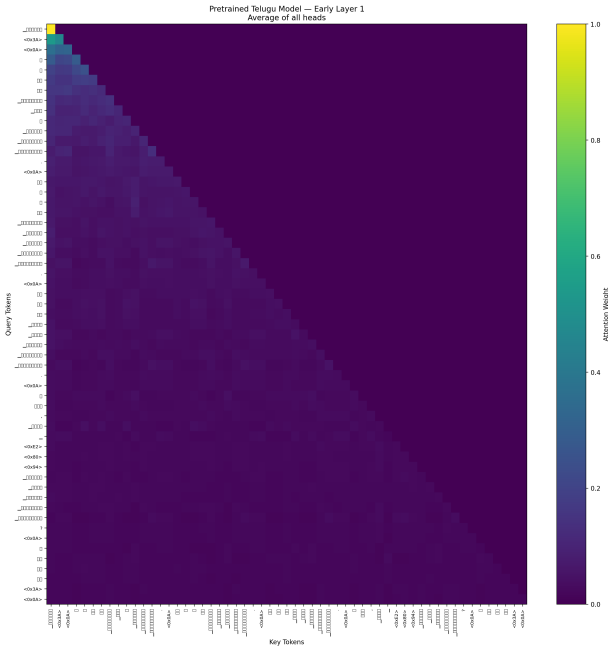
Pretrained v/s Finetuned Results

Template	Pretrained	Finetuned	Improvement
Overall	0.0	0.8175	0.8175
T1	0.0	0.565	0.565
T2	0.0	0.34	0.34
T3	0.0	1.0	1.0
T4	0.0	1.0	1.0
T5	0.0	1.0	1.0
T6	0.0	1.0	1.0

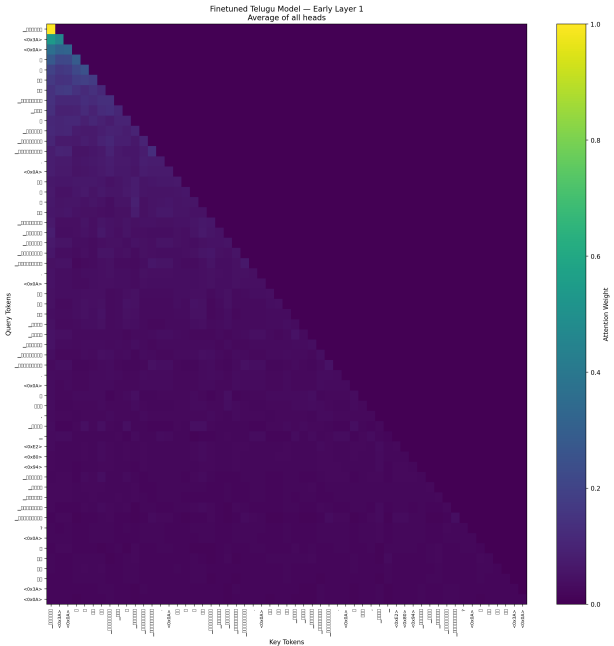
The model achieves an accuracy close to 1.0 on T3, T4, T5, and T6. This may be partly due to the similar structure of these templates, where the relationship between the entities often follows a consistent pattern, such as one entity being greater than another. In contrast, T1 and T2 require the model to perform more direct numerical reasoning and learn the specific comparison task. The substantial improvement on T1 and T2 compared with the initial pretrained model indicates that fine-tuning has improved the model’s ability to perform the required reasoning task.

Inferences

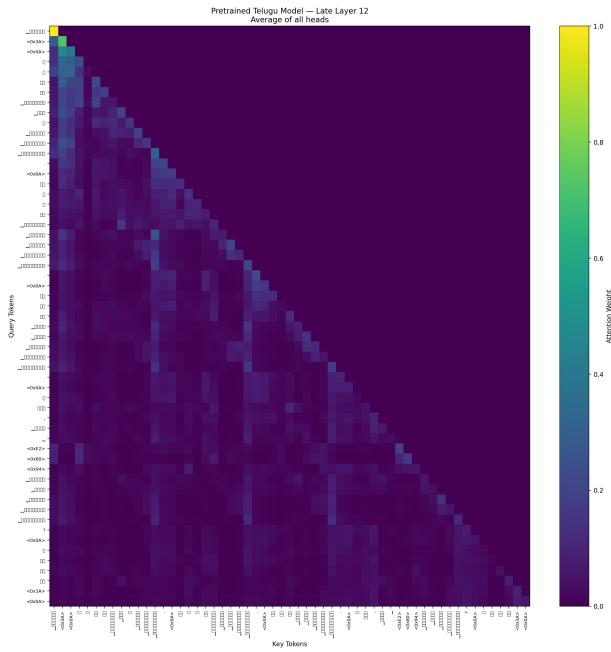
1. The Layer 1 heatmaps for the pretrained and fine-tuned models are almost identical.
2. In the pretrained Layer 12 heatmap, attention remains spread out with no clear focus. In contrast, in the fine-tuned Layer 12 heatmap, a sharp, bright stripe appears at one specific token near the end. This indicates that the fine-tuned model has learned to lock onto a token within the question line itself, rather than the earlier fact sentences where the actual comparisons are stated.
3. The entropy chart shows that both models behave similarly in the early layers, but the fine-tuned line drops lower than the pretrained line from approximately layer 7 onward. This means that the fine-tuned model’s attention becomes more focused and sharp in the later layers.
4. Furthermore, the fine-tuned model’s attention distance is shorter than the pretrained model’s in the late layers, rather than longer. This points to the problem in dataset generation that was highlighted in the previous section. The attention distance proves that the model pulls the fact from the question rather than from the provided facts.



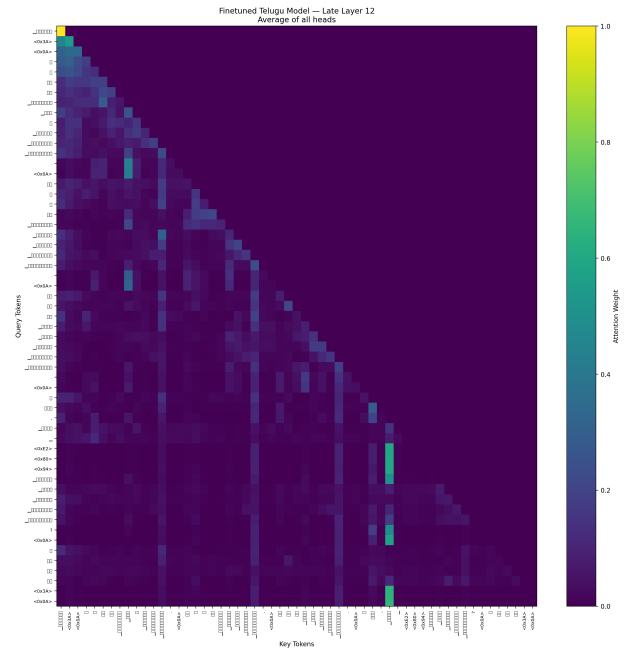
figureLayer 1 heatmap – Pretrained



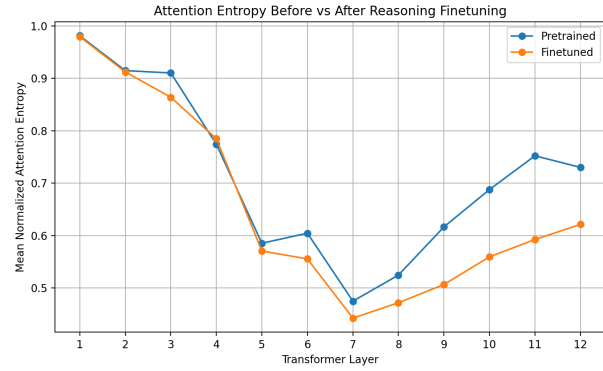
figureLayer 1 heatmap – Fine-tuned



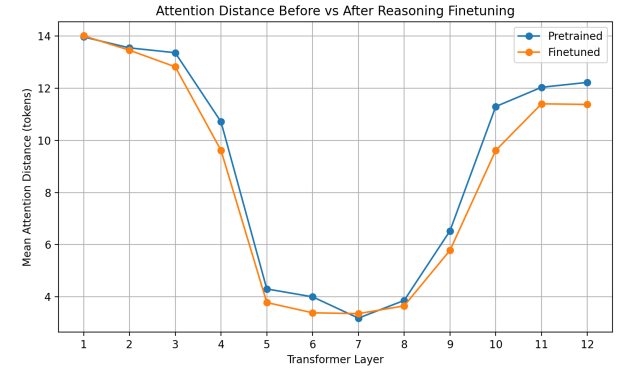
figureLayer 12 heatmap – Pretrained



figureLayer 12 heatmap – Fine-tuned



figureMean attention entropy by layer



figureMean attention distance by layer

Nepali Model Analysis

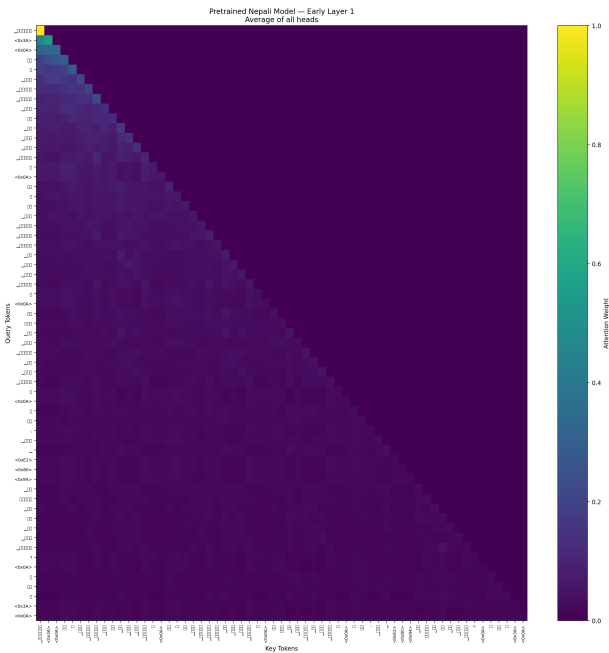
Pretrained v/s Finetuned analysis

Template	Pretrained	Finetuned	Improvement
Overall	0.0	0.80	0.80
T1	0.0	0.54	0.54
T2	0.0	0.315	0.315
T3	0.0	1.0	1.0
T4	0.0	1.0	1.0
T5	0.0	1.0	1.0
T6	0.0	1.0	1.0

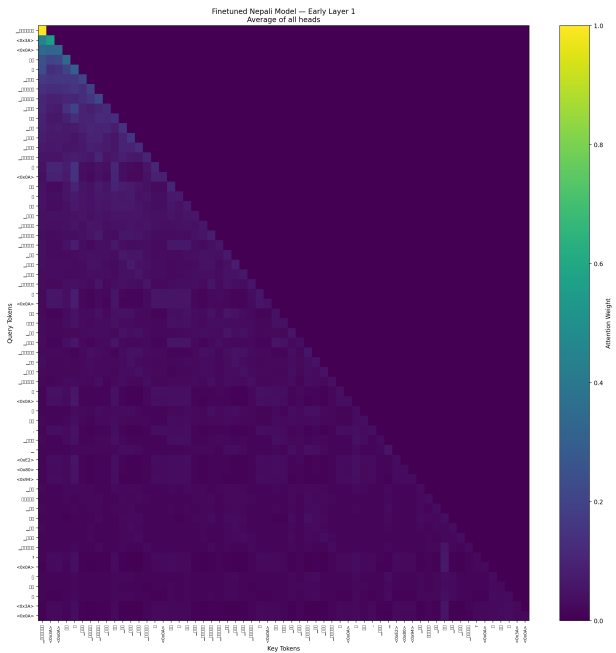
Attention Inferences

1. The Layer 1 heatmaps for the pretrained and fine-tuned models are almost identical, as seen in telugu.
2. In the pretrained Layer 12 heatmap, attention remains spread out with no clear focus, similar to the Telugu model. However, the fine-tuned Layer 12 heatmap behaves differently instead of a single sharp stripe at one token, attention is split across a few different spots in the question area.
3. The entropy chart shows a similar early pattern, with both models tracking closely through layer 4. The fine-tuned line then drops much lower indicating sharper focus, similar to the Telugu model. However, by layer 12, the fine-tuned line climbs back up to about 0.81, almost catching up to the pretrained line (0.90). Unlike the Telugu model, where fine-tuned attention remained clearly sharper through layer 12.
4. A key difference is observed in the distance chart: the fine-tuned attention distance goes higher than the pretrained distance from layer 6 onward, and the gap continues to grow, ending at approximately 15.8 tokens

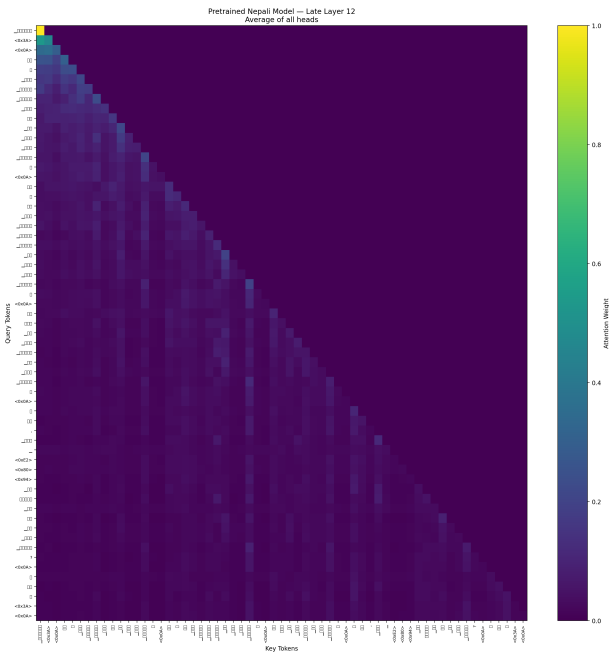
(fine-tuned) versus 13.1 tokens (pretrained) at layer 12. This is the exact opposite of the Telugu model, where the fine-tuned distance was lower than the pretrained distance in the late layers. On the surface, this longer distance suggests that the model is reaching further back possibly toward the fact sentences rather than staying anchored to a nearby token in the question.



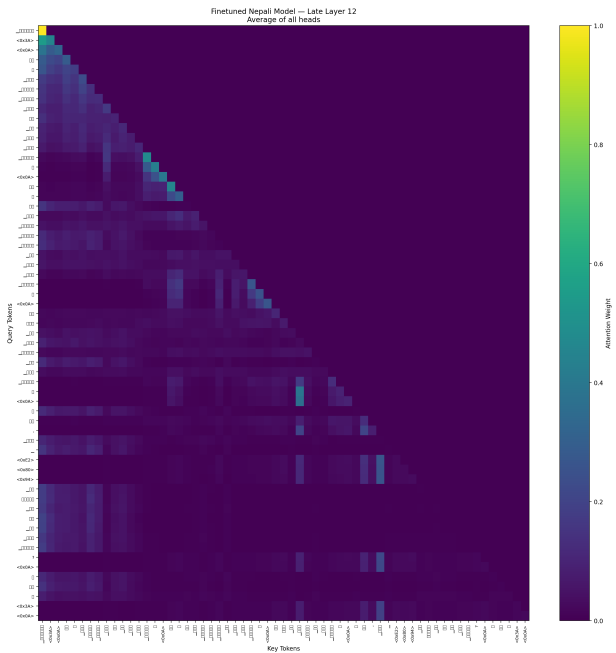
figureLayer 1 heatmap – Pretrained (Nepali)



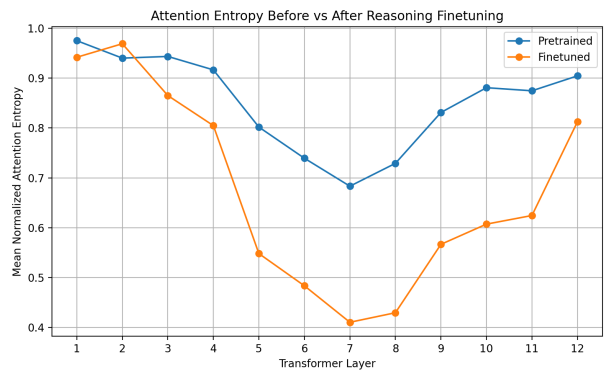
figureLayer 1 heatmap – Fine-tuned (Nepali)



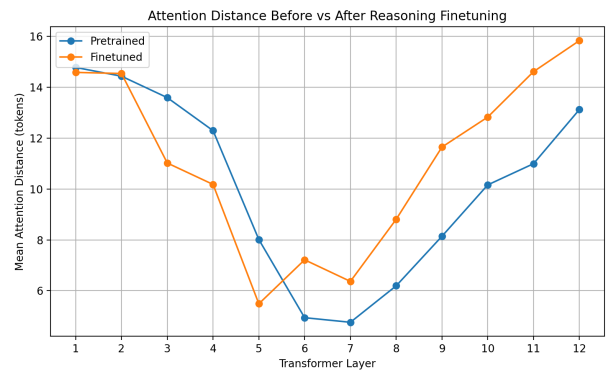
figureLayer 12 heatmap – Pretrained (Nepali)



figureLayer 12 heatmap – Fine-tuned (Nepali)



figureMean attention entropy by layer (Nepali)



figureMean attention distance by layer (Nepali)

Telugu vs Nepali Comparison

How did data scale and quality differ between Model H and Model L?

Telugu used 60.7M unique sentences (863.8M tokens) compared to Nepali's 25.6M sentences (554.1M tokens). Because Nepali required extensive crawling and lacked a dedicated human reviewer, it was difficult to filter out noisy or machine-translated text. Consequently, despite passing the same automated quality checks, the Nepali corpus likely retained more unverified, lower-quality data than Telugu.

How do language-modeling and reasoning results compare across the two resource tiers?

While Telugu's larger corpus yielded a better Language Modeling validation loss (5.02 vs. Nepali's ≈ 5.3), both models performed almost identically on the reasoning tasks. Nepali achieved 80.9% overall accuracy, closely matching Telugu's 81.75%, with both models scoring perfectly (100%) on templates T3–T6 and struggling similarly on T1 and T2. Furthermore, Nepali's fine-tuning loss converged more cleanly (≈ 0.15 at epoch 3) without the late-epoch validation uptick seen in Telugu. The identical perfect scores on T3–T6 confirm a positional shortcut in the dataset generator.

What tokenizer / corpus factors most affected the lower-resource model?

The tokenizer was not the issue Nepali's fertility (1.382) and UNK rate (0.0%) were comparable to or better than Telugu's. The real bottleneck was corpus composition. The lack of diverse, high-quality manual data and the heavy reliance on unverified, crawled web corpora.

What evidence explains the observed differences?

Loss curves show it directly. Nepali's validation loss plateaus higher (≈ 5.3) than Telugu's (5.02). Furthermore, the attention heatmaps reveal that Telugu's late layers recover longer-range attention more sharply (≈ 7.4 tokens by layer 9) than Nepali's (≈ 6.0 tokens by layer 11).